

So i am starting a Pulp tutorial , I am gonna write the first 10 minutes and you explain more to me (suggest a site that receives YouTube video links and explain what is inside).Pulp : Parallel ultra-low power one chip processing that targets energy efficiency , placed near edge of IOT sensors for more energy efficient operation. it brings computing power to the IOT devices, that is achieved by running the Microcontroller close to the threshold voltage of transistors in order to work at the optimum energy efficient point. However, working near the threshold voltage means that the voltage is low. thus, the operating frequency is low. Therefore, to meet constraints Multiple cores are used. Hence meeting performance at low power. Additionally, it is a versatile platform to target different sensors by just adjusting the software running on the MC).

1. Clarifying what you already said (conceptually)

What you wrote is **basically correct**, just let me tighten the mental model:

- **PULP (Parallel Ultra-Low Power)** targets **energy efficiency**, not peak performance.
- It is designed for **edge IoT devices**, meaning:
 - close to sensors
 - limited battery
 - often always-on
- The key idea:
Minimize energy per operation, not clock frequency.

Near-threshold operation (important intuition)

- Running **near threshold voltage**:
 - ☒ drastically reduces dynamic and leakage power
 - ☒ reduces maximum clock frequency
- This creates a problem: **low performance per core**

PULP's solution

Instead of:

- one fast core at high voltage

PULP uses:

- **multiple simple RISC-V cores**
- **running slowly**

- **in parallel**

So performance is recovered by **parallelism**, not frequency.

This is why PULP is:

- great for **signal processing**
- vision
- audio
- sensor fusion
- ML at the edge

Your explanation is good — just remember:

PULP trades **time** for **parallelism** to save **energy**.

2. Explaining the slide: *PULPissimo Architecture*

This slide shows **PULPissimo**, which is:

a **single-core reference SoC** in the PULP ecosystem
(used a lot for tutorials and as a control processor)

Let's break it down top-down.

A. Overall view

The green box is the **SoC (System on Chip)**.

Inside it you have:

- **Compute**
- **Memory**
- **Peripherals**
- **Interconnect**
- **Power & clock management**

All optimized for **low power**.

B. RISC-V Microcontroller

On the right side you see:

- **RISC-V core**
 - Executes your C / assembly code
 - Simple, efficient, predictable
- **INTC**
 - Interrupt controller
- **DEBUG**
 - JTAG support for debugging

This is your **brain**, but it's intentionally *not fancy*.

C. Memory system (very important in PULP)

1. L2 Memory (512 KB)

- Shared memory
- Used for:
 - code
 - data buffers
- Close to the core → low latency → low energy

2. L2 SCM Bank (16 KB)

- **Latch-based memory**, not SRAM
- Extremely energy-efficient
- Used for:
 - always-on tasks
 - ultra-low power modes

This is a **big deal** in PULP designs.

Memory access often costs more energy than computation — PULP optimizes memory first.

D. Logarithmic Interconnect

In the middle of the SoC:

- **Logarithmic Interconnect**
 - Connects cores, memories, accelerators
 - Scales well as you add more masters
 - Lower energy than a big shared bus

This is what allows **parallel cores + accelerators** to coexist efficiently.

E. HWCE – Hardware Convolution Engine

You see **HWCE** blocks.

- Specialized accelerator for:
 - convolutions
 - image processing
 - early ML workloads
- Offloads heavy math from the CPU
- Much more energy-efficient than software

This shows:

PULP is **heterogeneous** (CPU + accelerators)

F. μ DMA (Micro DMA) – extremely important

The vertical orange block: **μ DMA**

Purpose:

- Move data between:
 - peripherals
 - memory

- **Without waking up the CPU**

This enables:

- CPU sleeps
- Data still flows
- Massive power savings

Think:

“Let hardware move data while software sleeps.”

G. Peripherals (left side)

You have:

- SPI / QSPI (fast external memory)
- I2C
- I2S (audio, microphones)
- UART
- Camera Interface (CAMIF)
- SDIO
- GPIO

All connected to:

- μ DMA
- APB bus (low-power control bus)

This is why PULP is good for **sensor-heavy systems**.

H. Power & Clock Management

At the bottom:

- **CLK**
- **TIMER**

- **EVENT**
- **SOC CTRL**
- **FLLs (Frequency Locked Loops)**

Key idea:

- Different domains
- Dynamic voltage/frequency scaling
- Fine-grained power control

This is how near-threshold operation is *practically usable*.

1. μ DMA (Micro Direct Memory Access)

What μ DMA actually is

μ DMA in PULP is **not** a generic DMA like you see in ARM MCUs.

It is a **lightweight, autonomous data-movement engine** designed specifically for:

- ultra-low power
- fine-grained transfers
- sensor streaming

Formal definition:

μ DMA moves data between **peripherals and L2 memory** without CPU intervention.

Why μ DMA exists (core motivation)

In ultra-low power systems:

- **Waking the CPU costs more energy than moving data**
- Polling peripherals is extremely wasteful

μ DMA allows:

- CPU **sleep or clock-gated**
- peripherals **stream data continuously**
- memory filled automatically

This is essential for **near-threshold operation**.

How μ DMA works (architectural view)

μ DMA sits between:

- Peripherals (SPI, I2C, I2S, CAM, UART, etc.)
- L2 memory

Each peripheral has:

- **its own μ DMA channel**
- configured via memory-mapped registers

A transfer is:

1. CPU programs μ DMA registers:
 - source / destination address
 - size
 - direction
2. CPU goes idle or sleeps
3. μ DMA autonomously:
 - fetches peripheral data
 - writes into L2 memory
4. Event / interrupt generated when done

What makes μ DMA “micro”

Compared to classic DMA:

- No complex descriptors
- No cache coherency logic
- No virtual memory
- No burst-heavy logic

Instead:

- Very small hardware
- Low switching activity
- Optimized for **short, frequent transfers**

μ DMA is designed for *energy per byte*, not bandwidth per second.

μDMA vs CPU copy (energy perspective)

Task	Energy
CPU reads peripheral register	✗ high
CPU copies to memory	✗ high
CPU waits / polls	✗ very high
μDMA moves data	☑ very low
CPU sleeps	☑ ultra-low

This is why **μDMA is central to PULP**, not an optional feature.

Typical μDMA use cases

- Camera frame capture
 - Audio microphone streaming
 - SPI sensor reads
 - Continuous data logging
 - Feeding accelerators (HWCE)
-

2. PULP vs PULPissimo

Short answer

- **PULP** → a *research platform / architecture family*
- **PULPissimo** → a *specific SoC implementation*

But let's be precise.

3. What is PULP?

PULP is:

- an **open-source platform**
- for designing **parallel ultra-low power SoCs**

It includes:

- Multi-core clusters
- Memory hierarchies
- Interconnects
- Accelerators
- Power management techniques

PULP is **not one chip**.

Examples in the PULP ecosystem:

- PULPissimo
- GAP8 / GAP9
- Mr. Wolf
- HERO platform

Key PULP characteristics

- Near-threshold operation
- Multi-core parallelism
- Shared L1/L2 memory
- Tight coupling between cores
- Software-controlled power domains

4. What is PULPissimo?

PULPissimo is:

- a **single-core PULP SoC**
- used as:
 - reference design
 - control processor
 - tutorial platform

Why PULPissimo exists

Multi-core PULP is complex.

PULPissimo provides:

- Same memory system

- Same μ DMA
- Same peripherals
- Same power management
- But **only one RISC-V core**

So you learn:

- PULP software stack
- PULP memory model
- μ DMA programming
- Peripheral handling

Without:

- cluster synchronization
- multi-core scheduling
- barriers / locks

PULPissimo architecture summary

Feature	PULPissimo
Cores	1 RISC-V
μ DMA	☒
L2 memory	☒
Accelerators	Optional
Peripherals	Full
Power mgmt	☒
Target	Education, control, reference

5. PULPissimo vs Full PULP (Cluster-based)

Aspect	PULPissimo	Full PULP
Cores	1	4–16+

Aspect	PULPissimo	Full PULP
L1 memory	✗	☑ (TCDM)
Parallelism	✗	☑
Cluster	✗	☑
Synchronization	✗	☑
Energy per task	Low	Ultra-low

Think of it like:

- **PULPissimo** = control plane MCU
- **PULP cluster** = compute accelerator

In real chips (e.g. GAP8):

- PULPissimo-like core handles:
 - OS
 - peripherals
 - power
- PULP cluster handles:
 - compute bursts

6. One-sentence intuition (useful for exams/interviews)

- **μDMA**: *“Moves data so the CPU can sleep.”*
 - **PULP**: *“Energy-efficient computing via parallelism at low voltage.”*
 - **PULPissimo**: *“A single-core PULP SoC used as a reference and control processor.”*
-

the cluster is a master port of the logarithmic interconnect, the cluster is an accelerator for the single core pulp platform but this accelerator is programmable, the standard is 8 cores

1. “The cluster is a master port of the logarithmic interconnect”

This is **100% correct**, and it’s *architecturally important*.

What “master port” means here

On the **logarithmic interconnect (LIC)**:

- **Masters** can initiate transactions
- **Slaves** only respond

The **cluster** being a master means:

- It can **fetch data from L2**
- It can **write results back to L2**
- It does **not** depend on the single-core MCU to move data

This is why:

The cluster can run independently once launched.

In practice:

1. PULPissimo core configures the cluster
2. Cluster DMA pulls data from L2 → L1
3. Cluster computes
4. Results are written back to L2
5. Event sent to MCU

This is classic **accelerator semantics**.

2. “The cluster is an accelerator for the single-core PULP platform”

Yes — but with a **very specific meaning of accelerator**.

How PULP defines “accelerator”

Unlike:

- fixed-function accelerators (FFT, AES, CNN-only)

The PULP cluster is:

- **fully programmable**
- **general-purpose**
- **software-defined accelerator**

So:

The accelerator is *parallel RISC-V software*, not hardwired logic.

This is why PULP is flexible across:

- vision
- audio
- DSP
- ML kernels

3. Why the cluster is separate from the MCU

The separation is deliberate:

PULPissimo core	Cluster
Control-oriented	Compute-oriented
Always-on	Power-gated
Handles IO	Handles math
Low performance	High parallel throughput

The cluster:

- can be **entirely switched off**
- only powered during compute bursts

This is **burst computing**, essential for energy efficiency.

4. “Standard is 8 cores” — why 8?

This is not arbitrary.

Reasons for 8 cores

1. **Energy sweet spot**
 - Diminishing returns beyond ~8 cores at low voltage
2. **Interconnect complexity**
 - Logarithmic interconnect scales well to 8
3. **Memory pressure**

- 64 KB L1 shared among cores
- 4. **Software simplicity**
 - Barriers, forks, OpenMP-like models remain tractable

So 8 cores gives:

maximum energy efficiency per mm² for near-threshold operation

(Some chips use 4 or 16, but 8 is the reference.)

5. L1 Memory = TCDM (Tightly Coupled Data Memory)

From the slide:

64 kB of L1 memory

This is **not a cache**.

Key properties:

- Shared by all 8 cores
- Multi-banked
- Single-cycle access
- Deterministic latency

Why this matters:

- No cache coherence
- No unpredictable stalls
- Perfect for parallel DSP

This is one of PULP's biggest architectural wins.

6. Shared Instruction Cache (I\$)

Instruction side:

- One **shared instruction cache**
- Reduces:
 - memory footprint
 - power consumption

Most kernels:

- run same code on all cores
- only data differs

So shared I\$ is:

- more energy efficient
 - simpler than per-core I\$
-

7. Shared FPU (2 FPUs for 8 cores)

This often confuses people.

Why FPUs are shared

FPUs:

- expensive in area
- expensive in power
- rarely used 100% of the time

So PULP:

- shares 1 FPU among 4 cores
- arbitration handles access

This works because:

- many kernels are integer-heavy
- FP usage is bursty

Result:

better energy efficiency than per-core FPUs.

8. Cluster DMA (L2 ↔ L1)

This DMA is **different from μ DMA**.

Key differences

μDMA	Cluster DMA
Peripheral ↔ L2	L2 ↔ L1
IO-focused	Compute-focused
Works with sensors	Works with cluster
Used by MCU	Used by cluster

Cluster DMA allows:

- double buffering
- compute + transfer overlap
- no core stalling

This is how PULP sustains throughput at low frequency.

9. Event unit & barriers

Critical for parallelism.

The **event unit** provides:

- hardware barriers
- fast synchronization
- clock-gating when cores idle

So:

- cores that finish early can sleep
- no busy waiting

Again: energy-first design.

10. One clean mental model (use this)

PULPissimo core = manager

PULP cluster = programmable accelerator

L2 = shared memory

L1 = scratchpad for parallel compute

Execution model:

1. MCU prepares data
2. MCU launches cluster
3. Cluster DMA loads working set
4. 8 cores execute in parallel
5. Results written back
6. Cluster powers off

before that he was speaking about the RISCV core and the logarithmic interconnect and how it can adapt to AXI and APB, generally not a single language. Then here i do not quite understand, he is talking about a Misalignment memory extension as the 32 bits cannot accept addresses not divisible by 4, so the extension is essential to save cycles and for better performance.

1. First: baseline RISC-V rule (no extensions)

In **standard RISC-V**:

- A **32-bit load (lw)**
→ address **must be divisible by 4**
- A **16-bit load (lh)**
→ address **must be divisible by 2**

If you do:

```
lw x2, 2(x10)
```

that is **illegal** in standard RISC-V and will:

- trap, or
- be unsupported (implementation-dependent)

This rule exists because:

- it simplifies hardware
 - it guarantees single aligned memory access
-

2. What “misaligned access” actually means

A **misaligned access** is simply:

loading/storing a word at an address that is not naturally aligned to its size

Example:

- load 32 bits at address `0x0000_0002` ❌ misaligned
 - load 16 bits at address `0x0000_0001` ❌ misaligned
-

3. What XPULP misaligned extension does (important)

The **XPULP misaligned memory extension** allows the *same instruction*:

```
lw x2, 2(x10)
```

to **work**, even though the address is not divisible by 4.

⚠️ Key point

This is **not a new instruction**.

That’s why the slide says:

“Misaligned memory accesses (not ISA extension)”

The ISA syntax does **not change**.

What changes is the **load-store unit behavior**.

4. How the hardware actually handles it

Let’s use their example:

Load 32 bits at address `0x0000_0002`

Memory layout (byte-addressed):

```
0x0000_0000 | [ B0 B1 B2 B3 ]  
0x0000_0004 | [ B4 B5 B6 B7 ]
```

The word starting at `0x0000_0002` spans **two aligned words**.

What the XPULP load-store unit does:

1. Read aligned word at `0x0000_0000`
2. Read aligned word at `0x0000_0004`
3. Extract:
 - upper 16 bits from first word
 - lower 16 bits from second word
4. Pack them into a single 32-bit register

All of this happens:

- **inside the load-store unit**
 - transparently to software
-

5. Why this is useful (real reason)

This is *not* about convenience.

It's about **performance and code density**.

Without misaligned support (standard RISC-V)

To do the same thing manually:

```
lh    x2, 2(x10)
lh    x3, 4(x10)
slli  x3, x3, 16
or     x2, x2, x3
```

That is:

- 4 instructions
 - more register pressure
 - more energy
 - more cycles
-

With XPULP misaligned support

```
lw x2, 2(x10)
```

- 1 instruction
- fewer cycles
- less code
- less energy

This is **exactly what PULP optimizes for**.

6. Why this matters especially in PULP workloads

PULP targets:

- DSP
- vision
- audio
- packetized sensor data

These workloads often use:

- packed data formats
- 8-bit or 16-bit elements
- arrays not aligned to word boundaries

Example:

- pixels
- audio samples
- network packets
- feature vectors

Misaligned access support lets you:

- process packed data directly
 - avoid unpacking overhead
 - keep tight inner loops small and fast
-

7. Why this is *not* standard RISC-V

Because:

- it increases load-store unit complexity
- requires multiple memory accesses per load
- adds muxing and shifting logic

In **general-purpose CPUs**, that's a bad trade-off.

In **PULP**:

- memory is small
- access patterns are predictable
- energy per instruction matters more than peak throughput

So it's a **deliberate, domain-specific trade-off**.

8. Relationship to RISCVY core and interconnect

You mentioned earlier:

- RISCVY core
- logarithmic interconnect
- AXI / APB adaptation

This fits perfectly:

- Logarithmic interconnect can handle multiple memory accesses cleanly
- L1 TCDM is multi-banked → parallel accesses are cheap
- Deterministic latency → misaligned loads don't cause chaos

So this extension **assumes** a PULP-style memory system.

9. One-sentence takeaway (important)

The XPULP misaligned memory extension allows standard RISC-V load/store instructions to transparently handle unaligned addresses by performing multiple aligned memory accesses in hardware, reducing instruction count, cycles, and energy for packed-data workloads.

So here he was talking about how this extension allow data update and pointer update via single instruction unlike RISCV which used 2 instructions , then he spoke about an Immediate which is at most 12 bits(explain it), after that he said that there exist only instruction cache and no data cache. for the post increment he said that the we have 2 registers as destinations thus we need another port

1. Post-increment load/store: what actually changes

Standard RISC-V (baseline)

A load/store does **one thing only**:

```
lw rd, imm(rs1)
```

Meaning:

- compute $\text{address} = \text{rs1} + \text{imm}$
- load data $\rightarrow \text{rd}$
- **rs1 is unchanged**

So in array iteration you need:

```
lw x2, 0(x10)
addi x10, x10, 4
```

Two instructions:

- one for data
- one for pointer update

XPULP post-increment extension

The extension allows:

```
lw x2, 0(x10!)
```

Semantics:

- load data into `x2`
- update $\text{x10} = \text{x10} + \text{stride}$

So:

- **data register updated**
- **pointer register updated**
- in **one instruction**

This is classic DSP-style addressing.

2. Why this saves energy (not just instructions)

Saving instructions means:

- fewer fetches
- fewer decodes

- fewer pipeline transitions
- smaller instruction memory footprint

On PULP:

instruction fetch energy $\approx$ computation energy

So this is a **real** gain.

3. “Immediate is only 12 bits” — what this means

This refers to **RISC-V instruction encoding**.

In standard RISC-V load/store

The immediate field is:

- **12 bits**
- signed
- range:
 −2048 to +2047 bytes

So:

```
lw x2, 4096(x10)    // ✗ impossible
```

You *must* do:

```
addi x10, x10, 4096
lw    x2, 0(x10)
```

This limitation comes from:

- fixed 32-bit instruction width
- RISC-V choosing simplicity over flexibility

4. Register-register load/store (second extension)

XPULP adds a form like:

```
lw x2, (x10, x11)
```

Meaning:

- $\text{address} = \text{x10} + \text{x11}$
- offset is now **full 32-bit**, stored in a register

Why this matters:

- large arrays
- multi-dimensional indexing
- pointer arithmetic without extra `addi`

Again:

fewer instructions $\rightarrow$ less energy

5. Why this is *not* common in vanilla RISC-V

Because:

- it complicates decode
- it complicates load-store unit
- it breaks strict “load/store simplicity”

PULP accepts this because:

- cores are simple
- frequency is low
- memory system is deterministic

6. “Only instruction cache, no data cache” — very important

This is a **core PULP design choice**.

Why no data cache?

Because:

- multi-core shared memory
- scratchpad (L1 TCDM)
- deterministic access latency
- no coherence logic
- lower energy

Data accesses go to:

- L1 TCDM (cluster)
- L2 SRAM

Instruction access:

- read-only
- shared
- perfect for caching

Hence:

I\$ yes, D\$ no

This is why post-increment addressing is *even more valuable*:

- no cache to hide extra loads/stores
- every instruction matters

7. The “two destination registers” problem (important microarchitectural point)

The speaker is making a **hardware-level observation** here.

Standard load

```
lw rd, offset(rs1)
```

Writes:

- **one register** $\rightarrow$ rd

Post-increment load

```
lw rd, offset(rs1!)
```

Writes:

- $rd \leftarrow$ loaded data
- $rs1 \leftarrow$ updated pointer

That is **two register writes in one cycle**.

8. Why this requires an extra register file write port

A typical small RISC-V core has:

- 2 read ports
- **1 write port**

Post-increment needs:

- **2 write ports**
OR
- serialize the writes → slower execution

This is exactly what the slide means by:

“It requires extra write register file port or slower execution”

So PULP designers had a choice:

- add hardware (area + power)
- or accept multi-cycle instruction

Different XPULP cores choose differently.

9. Why PULP still accepts this trade-off

Because:

- these instructions appear in **tight inner loops**
- loops dominate energy
- saving instructions > extra port cost

Again:

optimize for **energy per loop**, not elegance.

10. Final mental summary (use this)

- **12-bit immediate**
→ encoding limitation of RISC-V

- **Register-register addressing**
→ bypass immediate limit
 - **Post-increment load/store**
→ combine memory + pointer update
 - **No data cache**
→ every instruction matters
 - **Two destination registers**
→ needs extra write port or extra cycle
-

RISCV does not have flags or branch delay slot (simple), Profile of DSP applications we have load compute where computational instructions depend on the kernel you are implementing then store and branch. this is for classical DSP application over a vector or a matrix, the shorter the loop the higher the branch price as the branch is take say for loop n it is taken n-1. So, aim was to avoid the branching overhead thus they used a HW loop extension with zero overhead. where instructions (or something) is known ahead so we know when to jump, no comparison. Initialization in the beginning is same for HW loop, but the other overhead pointer update is embedded in the HW loop. for nested loops inner most loops are done using HW loops but outer most is done through branches. then he spoke about the Area vs performance price and how speed up factor by 2 was the optimal point.

1. Why branches are expensive in PULP (context you already sensed)

RISC-V design facts

- No flags
- No condition codes
- No branch delay slot
- Simple pipeline

A loop like:

```
for (i = 0; i < N; i++) {
    body();
}
```

Becomes:

```
loop:
    body
    addi i, i, 1
    blt i, N, loop
```

This implies **every iteration**:

- comparison

- branch instruction
- possible pipeline flush
- instruction fetch redirection

In **DSP kernels**:

- loop body is *very short*
- branch cost is a **large fraction** of total cycles

The shorter the loop body, the higher the branch penalty per useful instruction.

You nailed that intuition.

2. DSP workload structure (why HW loops matter)

Typical DSP inner loop:

```
load
compute
store
branch
```

For small kernels:

- compute = 1–3 instructions
- branch = 1 instruction
- branch penalty $\approx$ 25–50% overhead

This is unacceptable in an **energy-driven architecture** like PULP.

3. Hardware loop (HWL / Zero-Overhead Loop): core idea

Fundamental idea

Replace **dynamic branch decisions** with **static loop control hardware**.

Key properties:

- loop bounds known *before* entering the loop
- no comparison inside the loop
- no branch instruction in the loop body
- PC redirection handled in hardware

So:

- no branch instruction fetched
- no misprediction
- no pipeline flush

Hence:

Zero overhead per iteration

4. How XPULP HW loops actually work (important)

This is the most misunderstood part.

HW loop = 3 things stored in special registers

For each loop level L :

1. **START_REG[L]**
→ first instruction of loop body
2. **END_REG[L]**
→ last instruction of loop body
3. **COUNT_REG[L]**
→ number of iterations

These are written **once**, before the loop starts.

What happens at runtime

- PC increments normally
- When PC reaches `END_REG[L]`:
 - hardware decrements `COUNT_REG[L]`
 - if `count ≠ 0` → `PC ← START_REG[L]`
 - else → exit loop

- ⚠ No compare instruction
- ⚠ No branch instruction
- ⚠ No flags

Just **PC control logic**.

This matches exactly what the speaker said:

“instructions are known ahead so we know when to jump, no comparison”

5. “Initialization is the same as normal loops”

Correct.

You still need:

- loop counter setup
- pointer setup
- register initialization

But this happens **once**, outside the loop.

The savings are inside the **hot loop**, which dominates execution time.

6. Pointer update being “embedded” — small correction

Pointer update is **not automatic** inside HW loops by default.

What actually happens is:

- HW loops remove **branch + compare**
- Pointer update is often combined with:
 - post-increment load/store
 - address auto-update extensions

So together:

- no branch instruction
- no explicit pointer `addi`

This is why XPULP extensions **stack well together**.

7. Nested loops: why only inner loops use HW loops

This is a **hardware cost decision**, not a limitation of theory.

XPULP implementation:

- Only **2 HW loop register sets**
 - $L = 0$
 - $L = 1$

So:

- inner loop $\rightarrow$ HW loop
- maybe second inner $\rightarrow$ HW loop
- outer loops $\rightarrow$ software branches

Why this is fine:

- inner loops execute most frequently
- outer loops are less performance-critical

Optimize where the cycles are.

8. Why speedup is capped at $\sim 2\times$

This is crucial and often misunderstood.

Best-case scenario

If loop body is:

```
1 instruction  
1 branch
```

Removing branch:

- execution halves
- speedup $\approx 2\times$

If loop body is larger:

```
5 instructions + 1 branch
```

Speedup:

- from 6 $\rightarrow$ 5 instructions
- $\approx 1.2\times$

So:

Speedup depends on loop body size.

The slide saying:

“Speedup can be up to factor 2”

means:

- **upper bound**
- **theoretical maximum**
- **typical DSP inner loops**

9. Area vs performance trade-off (why stop here)

Hardware loops cost:

- PC comparison logic
- counters
- registers
- control FSM

More HW loops:

- more area
- more leakage
- diminishing returns

PULP designers observed:

- 1–2 HW loops → big gains
- more than that → little benefit

So:

~2× speedup at minimal area cost = optimal energy point

This is *classic PULP philosophy*.

10. Final mental summary (clean and correct)

- RISC-V has no flags, no delay slots → branches are expensive
 - DSP inner loops are short → branch dominates cost
 - HW loops remove:
 - compare
 - branch
 - pipeline disruption
 - Loop control is done by hardware using:
 - start PC
 - end PC
 - iteration counter
 - Only inner loops use HW loops
 - Speedup is bounded ($\sim 2\times$)
 - Area vs energy trade-off is explicitly optimized
-

He did not say much here, he specified that bit manipulation is on bit level or a group of bits and that they are useful for software interrupts

1. Big picture: why bit manipulation exists at all

Bit manipulation instructions operate on:

- **single bits**
- or **bit fields** (groups of bits)

They are not about math throughput — they are about:

- control
- state
- packing/unpacking information efficiently

This is *very different* from MACs, FP, or vector ops.

2. Why baseline RISC-V was weak here (historically)

Early RISC-V deliberately avoided:

- complex bitfield ops

- popcount
- rotate
- bit scan

Reason:

- keep the base ISA minimal
- let extensions evolve later

So common operations required **long instruction sequences**.

Example: population count (count number of 1s):

- 10–20 instructions in plain RISC-V
- 1 instruction with bitman extension

For PULP, this mattered because:

control-heavy code still runs a lot on ultra-low-power MCUs.

3. RVB: the “official” RISC-V bitman extension

RISC-V reserved the **RVB** (bit manipulation) extension space:

- but at the time of PULP design
- it was **not finalized**

So PULP:

- implemented **XPULP bitman instructions**
- contributed ideas back to the RISC-V community
- planned future alignment with RVB

This explains the slide wording:

“possibly will align with RVB”

4. What XPULP bit manipulation instructions actually do

Let’s go through the list in *functional terms*.

4.1 Extract N bits starting from M

Example use:

- unpack a field from a register
- decode status words
- extract sub-pixels / packed data

Instead of:

```
srl  
andi
```

You get:

- **one instruction**
- optional sign extension

This is extremely common in:

- control registers
 - protocol headers
 - interrupts status words
-

4.2 Insert N bits starting from M

Used when:

- updating a field inside a control register
- modifying configuration words

Without this:

- mask
- shift
- OR
- multiple instructions

With XPULP:

- single instruction
 - fewer read-modify-write cycles
-

4.3 Clear / set N bits starting from M

Typical use:

- enable/disable features
- acknowledge interrupts
- toggle flags

This is **core MCU code**, not DSP math.

5. Find first / last bit set (bit scan)

These instructions answer:

- “Which event happened first?”
- “Which interrupt is pending?”
- “Which resource is free?”

Very common in:

- interrupt controllers
- schedulers
- RTOS kernels
- event units (like PULP’s)

Without hardware support:

- loop
- test bit by bit
- branch heavy
- slow + energy expensive

With bit scan:

- $O(1)$
 - predictable latency
-

6. Popcount (count number of 1s)

Used for:

- Hamming weight

- masks
- voting
- error detection
- sparse data analysis

The slide example shows exactly this:

Original RISC-V

- loop
- shift
- mask
- add
- branch

XPULP

`p.cnt x8, x8`

One instruction.

7. Rotate instructions

Rotate $\neq$ shift.

Used in:

- cryptography
- hash functions
- checksums
- circular buffers

Without rotate:

- shift left
- shift right
- OR
- 3 instructions minimum

Rotate:

- 1 instruction
- lower latency
- lower energy

8. Why “useful for software interrupts” specifically

This is an important remark from the speaker.

Software interrupts & control code rely on:

- bit masks
- status registers
- event flags
- enable/disable bits

Typical flow:

1. Read interrupt status register
2. Find which bit is set
3. Clear that bit
4. Jump to handler

With bitman instructions:

- fewer instructions
- faster ISR entry/exit
- lower interrupt latency
- less energy per interrupt

On always-on IoT nodes:

interrupts happen **a lot**.

So bitman pays off.

9. Why this fits PULP philosophy perfectly

Bit manipulation instructions:

- do **not** increase clock frequency
- do **not** increase datapath width
- add **small combinational logic**
- save many instructions

This is:

low area → high energy efficiency

Exactly what PULP wants.

10. Final mental summary (tie it all together)

- Bit manipulation $\neq$ DSP math
- It targets:
 - control
 - interrupts
 - status handling
 - packed data
- Baseline RISC-V was intentionally minimal
- XPULP added what real MCUs need
- Many of these ideas later appeared in official RVB

To boost computation, As there is MAC we need 3 reading ports now, absolute is subtraction and comparison and have nearly zero cost.

1. Why DSP extensions exist in XPULP (context)

Baseline RISC-V is:

- integer-centric
- scalar
- not optimized for signal processing inner loops

But PULP targets:

- FIR / IIR filters
- convolutions
- dot products
- correlation
- audio & vision kernels

All of these are dominated by:

Multiply–Accumulate (MAC) in tight loops

So XPULP adds **DSP-centric instructions** instead of raising frequency or adding big FP units.

2. MAC: why it needs *three* read ports

What a MAC does

A MAC instruction conceptually computes:

$$\text{acc} = \text{acc} + (\text{a} * \text{b})$$

Operands:

1. accumulator (old value)
2. multiplicand a
3. multiplicand b

That's **three source values**.

Baseline RISC-V register file

Typically:

- 2 read ports
- 1 write port

Enough for:

```
add rd, rs1, rs2
```

MAC problem

A single-cycle MAC wants:

- read `acc`
- read `a`
- read `b`

So:

3 read ports are required

This is exactly what you noted.

Architectural choices in XPULP

To support MAC efficiently, XPULP may:

- add a **third read port**, or
- pipeline the operation (multi-cycle), or
- restrict which register is the accumulator

Different PULP cores balance:

- area
- power
- throughput

But the **key idea** is:

MAC is expensive mainly because of register file pressure, not arithmetic.

3. Why MAC is such a big win anyway

Without MAC:

```
mul  t0, a, b
add  acc, acc, t0
```

2 instructions
2 fetches
2 decodes
2 register writes

With MAC:

```
mac acc, a, b
```

1 instruction
Half the control overhead
Lower energy per operation

In tight loops, this dominates everything else.

4. ABS instruction: why it's "almost free"

You said:

absolute is subtraction and comparison and have nearly zero cost

That's **exactly right**.

Absolute value definition

$\text{abs}(x) = (x < 0) ? -x : x$

Hardware reality

In an ALU you already have:

- adder/subtractor
- sign bit
- comparator

So ABS can be implemented as:

- check MSB
- optionally invert + add 1

This **reuses existing ALU hardware**.

That's why the slide says:

"ABS reuses the adder and comparator in the ALU"

Area cost $\approx$ control logic only

Energy cost $\approx$ negligible

5. CLIP / Saturation: slightly more expensive, still cheap

What clipping does

$\text{if } (x > \text{max}) \ x = \text{max}$

$\text{if } (x < \text{min}) \ x = \text{min}$

Requires:

- two comparisons
- a mux

- no heavy arithmetic

XPULP:

- adds **one extra comparator**
- reuses existing ones where possible

Still:

- tiny area increase
 - big software simplification
-

6. MIN / MAX: same story as ABS

MIN / MAX:

- comparison + mux
- no new datapath

Very common in:

- DSP
- ML activation functions (ReLU)
- normalization

Again:

control logic heavy, arithmetic light

Perfect for PULP.

7. Fixed-point support: normalization & rounding

This is subtle but important.

Fixed-point arithmetic needs:

- scaling
- normalization
- rounding

Instead of doing:

add
shift
mask
add

XPULP:

- **connects adder output directly to shifter**
- exploits multi-operand adders for rounding

So:

- no extra pipeline stages
- no extra functional units
- smart wiring

This is *classic low-power hardware design*.

8. Why DSP extensions fit PULP philosophy

DSP extensions:

- increase **work per instruction**
- reduce loop body length
- reduce instruction fetch energy
- avoid increasing clock frequency

PULP prefers:

“Do more per cycle at low frequency”

Not:

“Run faster with bigger power cost”

9. One-line explanation of the slide (exam/interview ready)

- **MAC** needs 3 read ports → main hardware cost
- **ABS / MIN / MAX / CLIP** reuse ALU blocks → near-zero cost
- **Fixed-point ops** reuse shifters & adders via smart wiring
- Net result: **large DSP speedup with minimal area/power**

10. Final takeaway

XPULP DSP extensions boost computational density in DSP kernels by adding MAC and fixed-point operations; MAC stresses the register file (3 reads), while ABS and similar ops are almost free because they reuse existing ALU hardware—making them ideal for ultra-low-power parallel processing.

For same register file , more than one operand can be stored inside register , more extensions can be added(the table below) where they can take more than one operand, second slide emphasizing the concept , he said we still reading same operands what changes is the mathematical operation and the carry chain (What SIMD stands for), in Riscv there are no flags so to check for example for overflow we have to do it in software.

1. What SIMD stands for

SIMD = Single Instruction, Multiple Data

Meaning:

One instruction operates on **multiple independent data elements** in parallel.

Key property:

- same opcode
- same control
- multiple data lanes

Classic examples:

- SSE / NEON / AVX
 - GPU SIMD lanes
 - Packed DSP operations
-

2. What *packed-SIMD* means in XPULP

XPULP uses **packed-SIMD (pSIMD)**, not vector registers.

Packed-SIMD idea

- A **single 32-bit integer register** contains:
 - **4 × 8-bit elements**, or
 - **2 × 16-bit elements**, or
 - **1 × 32-bit element**

Example:

```
rs1 = [ b3 | b2 | b1 | b0 ]  
rs2 = [ c3 | c2 | c1 | c0 ]
```

One instruction performs:

```
rD[i] = rs1[i]  $\oplus$  rs2[i]    for all i
```

That's SIMD **inside one register**.

3. “Same register file” — very important point

You are **100% correct** here.

What does NOT change

- Same integer register file
- Same number of registers
- Same read/write ports
- Same physical storage

What DOES change

- **How the bits are interpreted**
- **How the ALU operates on them**

The instruction encoding tells the hardware:

- treat register as bytes (`.b`)
- treat register as halfwords (`.h`)
- treat register as word (`.w`)

So:

We still read the same operands — only the *semantics* change.

This is exactly what the second slide illustrates.

4. Understanding the example (slide 2)

Registers:

```
rs1 = 0x03 02 01 00
rs2 = 0x0D 0C 0B 0A
```

Normal RISC-V add

```
add rD, rs1, rs2
```

Meaning:

```
rD = 0x03020100 + 0x0D0C0B0A
```

Single 32-bit addition.

Packed halfword add

```
add.h rD, rs1, rs2
```

Meaning:

```
rD[0] = 0x0100 + 0x0B0A
rD[1] = 0x0302 + 0x0D0C
```

Two **independent 16-bit adds**.

Packed byte add

```
add.b rD, rs1, rs2
```

Meaning:

```
rD[0] = 0x00 + 0x0A
rD[1] = 0x01 + 0x0B
rD[2] = 0x02 + 0x0C
rD[3] = 0x03 + 0x0D
```

Four **independent 8-bit adds**.

This is **SIMD**.

5. Carry chain control (this is the key hardware trick)

A normal 32-bit adder:

- propagates carry across all bits

For packed-SIMD:

- **carry must be blocked between lanes**

XPULP achieves this by:

- inserting carry-select / carry-mask logic
- controlling carry-in per sub-lane

Examples from slide 3

Byte subtraction

Carry = 1, 1, 1, 1

Each byte:

- independent subtraction
- no borrow propagation

Halfword subtraction

Carry = c0, 1, c0, 1

Carries propagate **within** halfwords, not across them.

Word subtraction

Carry = c0, c0, c0, 1

Full 32-bit behavior.

This is **why pSIMD is cheap**:

- same adder
 - smart carry gating
 - no duplication of ALUs
-

6. Why this is different from RVV (vector extension)

pSIMD (XPULP)	RVV (RISC-V Vector)
---------------	---------------------

Packed in integer regs	Separate vector registers
------------------------	---------------------------

Fixed width (32b)	Variable length
-------------------	-----------------

Low area	High area
----------	-----------

Embedded / DSP	HPC / servers
----------------	---------------

Simple control	Complex control
----------------	-----------------

XPULP explicitly says:

“RVV is for high performance, pSIMD is for embedded systems”

7. About flags and overflow (important correction)

You said:

“in RISC-V there are no flags so to check overflow we have to do it in software”

This is **correct**, but let’s be precise.

RISC-V has:

- no condition flags
- no overflow flag
- no carry flag

So:

- overflow detection must be done with:
 - comparisons
 - extra instructions
 - bit tricks

Packed-SIMD implication

- each lane can overflow independently
- hardware does **not** raise flags

- software must:
 - use saturation instructions
 - or explicit checks

This is why:

- CLIP / saturation DSP instructions are crucial
 - bit manipulation + DSP + pSIMD work together
-

8. Why packed-SIMD fits PULP perfectly

Packed-SIMD:

- increases **operations per instruction**
- reuses existing hardware
- avoids large vector units
- works well at low frequency
- ideal for:
 - image pixels
 - audio samples
 - neural network int8 kernels

This is **energy-efficient parallelism**.

9. One-paragraph summary (exam / presentation ready)

XPULP packed-SIMD implements SIMD by packing multiple 8- or 16-bit data elements inside standard 32-bit integer registers. The same register file and ALU are reused; only the instruction semantics and carry-chain behavior change, allowing independent per-lane operations. Unlike RVV vector extensions, packed-SIMD is fixed-width, low-overhead, and optimized for embedded DSP workloads. Since RISC-V has no flags, overflow and saturation are handled explicitly via DSP instructions rather than hardware condition codes.

Still same context here he is speaking about the shuffling (shuffling register contents and store them in a destination register) where two reg have the actual data and third one tells them where to go (Mask register) so need 3 read ports

1. Why shuffle instructions are needed (big picture)

Packed-SIMD only works **if corresponding elements line up**:

```
byte 0 with byte 0  
byte 1 with byte 1  
...
```

But real data often arrives:

- interleaved
- misaligned
- from different registers
- in memory-friendly rather than compute-friendly layout

So before SIMD math:

you must rearrange data inside registers

That is exactly what **shuffle instructions** do.

2. What “shuffle” means here

A **shuffle instruction**:

- takes **bytes (or halfwords)** from one or more source registers
- **reorders** them
- writes the recombined vector into a destination register

This is *not* arithmetic — it’s **data movement + selection**.

3. The three registers involved

From the instruction:

```
pv.shuffle2.b rD, rA, rB
```

Roles:

- **rA** → data source register
- **rD (old value)** → optional secondary data source
- **rB** → *mask register* (controls routing)

This is why:

two registers contain actual data, the third controls where it goes

Your summary is spot-on.

4. Understanding the mask register (rB)

Each **byte lane** in rD is controlled by:

- a **2-bit or 3-bit field** in rB

From the slide:

```
rD[3] = (rB[26] == 0) ? rA[rB[25:24]] : rD[rB[25:24]]
rD[2] = (rB[18] == 0) ? rA[rB[17:16]] : rD[rB[17:16]]
rD[1] = (rB[10] == 0) ? rA[rB[ 9: 8]] : rD[rB[ 9: 8]]
rD[0] = (rB[ 2] == 0) ? rA[rB[ 1: 0]] : rD[rB[ 1: 0]]
```

Interpretation:

- For each output byte:
 - choose **which input register** (rA or old rD)
 - choose **which byte index**
- All routing decisions are encoded in **rB**

So rB is effectively a **byte-level crossbar controller**.

5. Diagram intuition (right side of slide)

Visually:

- rA provides bytes w10-w13
- old rD provides bytes w00-w03
- rB selects:
 - source register
 - source byte
- destination rD is filled accordingly

Result:

```
rD = [ w10 | w03 | w02 | w01 ]
```

All in **one instruction**.

6. Why this needs 3 read ports

Let's list what hardware must read **in the same cycle**:

1. **rA** → source data
2. **old rD** → second source data
3. **rB** → mask / routing info

That's **three independent register reads**.

Baseline RISC-V:

- 2 read ports

Shuffle:

- **3 read ports required**

So your statement:

“so need 3 read ports”

is **architecturally exact**.

7. Why PULP accepts this cost

Because shuffle:

- appears in **inner DSP loops**
- replaces many:
 - shifts
 - masks
 - ORs
- drastically reduces instruction count

Energy trade-off:

- - one RF read port
- – many instructions, fetches, branches

Net result:

lower energy per useful operation

Classic PULP design logic.

8. Why this is essential for packed-SIMD

Packed-SIMD arithmetic alone is **not enough**.

You also need:

- pack
- unpack
- interleave
- de-interleave
- transpose small vectors

Shuffle is what makes pSIMD **usable**, not just powerful.

9. Relationship to other architectures

- ARM NEON → `tbl`, `zip`, `uzp`
- x86 SIMD → `pshufb`
- GPUs → lane swizzles

XPULP's shuffle is the **minimal embedded version** of the same idea.

10. Final one-paragraph summary

XPULP shuffle instructions rearrange packed data inside 32-bit registers to align elements for SIMD computation. Two registers provide the data sources, while a third register acts as a mask that encodes per-lane routing decisions. Because the hardware must read both data operands and the mask simultaneously, shuffle instructions require three register-file read ports. This small hardware cost enables efficient data alignment and makes packed-SIMD practical for real DSP workloads.

this a comprehensive example showing how the code shrinks and the gain in performance

The high-level C code (what we want)

```
for (i = 0; i < 100; i++)  
    d[i] = a[i] + b[i];
```

This is the *canonical DSP kernel*:

- streaming loads
- simple compute
- streaming store
- tight loop

Perfect for PULP.

1. Baseline RISC-V (11 cycles / output)

What happens per iteration

You need:

- load `a[i]`
- load `b[i]`
- add
- store
- increment **3 pointers**
- decrement loop counter
- branch

So the loop body is dominated by **overhead**, not math.

Key costs

- explicit pointer updates (`addi`)
- branch every iteration
- single-element processing

➡ **11 cycles per output**

This is your reference point.

2. Auto-increment load/store (8 cycles / output)

What changed

- Pointer update is **fused into load/store**
- Fewer `addi` instructions

Now each iteration:

- loads already advance pointers
- one less instruction per pointer

What's still expensive

- branch every iteration
- scalar (1 element per loop)

 **8 cycles per output**

This shows:

Address-generation instructions are real overhead.

3. Hardware loop (5 cycles / output)

What changed

- Branch removed from the loop body
- Loop control handled by HW loop registers

So now:

- no compare
- no branch
- no pipeline disruption

The loop becomes:

```
load
load
add
store
```


Pure work.

→ 5 cycles per output

This is a *huge* win for short loops.

4. Packed-SIMD + HW loop (1.25 cycles / output)

This is where everything comes together.

What changed

- **Packed-SIMD** processes **4 bytes at once**
- Same loop body, but each iteration produces **4 outputs**
- Still no branch
- Still auto-increment loads

So per loop iteration:

- 2 vector loads
- 1 vector add
- 1 vector store
- loop overhead = 0

Why 1.25 cycles?

Because:

- ~5 cycles per *iteration*
- each iteration produces **4 outputs**

$$5 / 4 = 1.25 \text{ cycles per output}$$

→ ~9× speedup vs baseline

Why this slide is so important

This is not about *one* extension.

It proves that XPULP extensions are:

- **orthogonal**
- **composable**
- **energy-driven**

Each layer removes a different class of overhead:

Extension	Removes
Auto-increment address update	Auto-increment address update overhead
HW loops	branch + compare overhead
Packed-SIMD	instruction-per-element limit

Together:

Almost all cycles go to *useful computation*.

The deeper architectural message

Notice what **did NOT change**:

- clock frequency
- pipeline depth
- cache hierarchy
- memory bandwidth

Performance came from:

- **doing more per instruction**
- **removing control overhead**
- **data-parallel execution**

That's classic near-threshold, energy-first design.

One-sentence takeaway (perfect exam answer)

This example shows how XPULP ISA extensions progressively remove loop, address, and scalar-processing overhead, shrinking code size and reducing cycles per output from 11 to 1.25 by combining auto-increment memory access, hardware loops, and packed-SIMD execution.

Yes — this is the perfect way to *cement* the whole lecture 🍌

I'll write **clean, representative assembly-style code** for **each layer**, matching exactly what the slide is illustrating.

We'll assume:

- x10 → pointer to a
- x11 → pointer to b
- x12 → pointer to d
- x4 → loop counter
- byte data (lb/sb)
- simple in-order core

Target C code:

```
for (i = 0; i < 100; i++)  
    d[i] = a[i] + b[i];
```

1 Baseline RISC-V (11 cycles / output)

No extensions. Everything explicit.

```
li    x4, 100          # loop counter  
li    x5, 0            # i = 0  
  
Lstart:  
    lb    x2, 0(x10)    # load a[i]  
    lb    x3, 0(x11)    # load b[i]  
    add   x2, x2, x3     # a[i] + b[i]  
    sb    x2, 0(x12)    # store d[i]  
  
    addi  x10, x10, 1    # a++  
    addi  x11, x11, 1    # b++  
    addi  x12, x12, 1    # d++  
  
    addi  x5, x5, 1      # i++  
    blt   x5, x4, Lstart # branch
```

Why it's slow

- 3 pointer updates
 - loop counter update
 - branch every iteration
 - 1 output per iteration
-

2 Auto-increment load/store (8 cycles / output)

Pointer update is **fused into memory ops**.

```
li    x4, 100
li    x5, 0

Lstart:
lb     x2, 1(x10!)    # load a[i], a++
lb     x3, 1(x11!)    # load b[i], b++
add    x2, x2, x3
sb     x2, 1(x12!)    # store d[i], d++

addi   x5, x5, 1
blt    x5, x4, Lstart
```

What improved

- pointer `addi` instructions removed
 - same semantics
 - still scalar
 - still branching
-

3 Hardware loop (5 cycles / output)

Branch + compare removed.

Loop count handled by **HW loop registers**.

```
lp.setupi 100, Lend    # HW loop: 100 iterations

Lstart:
lb     x2, 1(x10!)
lb     x3, 1(x11!)
add    x2, x2, x3
sb     x2, 1(x12!)
```

Lend:

Why this is big

- **no branch**
 - **no compare**
 - no loop-counter instructions
 - only useful work in the loop
-

4 Packed-SIMD + HW loop (1.25 cycles / output)

Now we process **4 bytes per iteration**.

Assume:

- `pv.add.b` → packed 8-bit SIMD add
- `lw` loads 4 bytes
- pointers advance by 4

```
lp.setupi 25, Lend      # 100 elements / 4 = 25 iters
```

Lstart:

```
lw      x2, 4(x10!)    # load 4 bytes from a
lw      x3, 4(x11!)    # load 4 bytes from b
pv.add.b x2, x2, x3     # 4 parallel adds
sw      x2, 4(x12!)    # store 4 results
```

Lend:

Why it's fast

- HW loop → zero loop overhead
- auto-increment → zero pointer overhead
- packed-SIMD → 4 outputs per iteration

5 cycles / iteration ÷ 4 outputs = 1.25 cycles/output

One compact comparison table

Version	Outputs / iter	Branch	Pointer add	Cycles / output
Baseline	1	✓	✓ ✓ ✓	11
Auto-inc	1	✓	X	8
HW loop	1	X	X	5
pSIMD + HW 4		X	X	1.25

Final architectural insight

Nothing here relied on:

- higher clock
- deeper pipeline
- caches
- speculation

All speedup came from:

- **ISA expressiveness**
- **control overhead elimination**
- **data-level parallelism**

That's why PULP works so well at **near-threshold voltage**.

Sharing as much as possible, no timing problems, alu for fixed point , alu for saturation and clipping

1. What this slide is really about

At the ISA level you saw:

- DSP ops
- saturation / clipping
- packed-SIMD
- rounding
- normalization

This slide answers the question:

How do we implement all that without blowing up area, power, or timing?

Answer:

One carefully shared ALU datapath, reused in different modes.

2. “Advanced ALU for XPULP extensions”

This is **not**:

- multiple independent ALUs
- one unit per instruction class

Instead:

- **one main datapath**
- multiple *views* of the same hardware
- controlled by `alu_operator`

That’s the core PULP philosophy.

3. Optimized datapath to reduce resources

Look at the right-hand diagram: it is **vertically partitioned**, not duplicated.

You can identify five logical blocks (colored):

1. **Vectorial adder path**
2. **Vectorial comparator / clip path**
3. **Bit manipulation unit**
4. **Logic unit**
5. **Shifter unit**

Crucially:

- operands A, B, C are *fanned out*
- results are multiplexed at the end
- only **one path is active per instruction**

So:

sharing happens in space (hardware reuse), not in time.

4. Multiple adders for rounding (important DSP detail)

Why rounding is hard

Rounding fixed-point results often needs:

```
result = (x + bias) >> shift
```

Instead of:

- adding
- then separately rounding
- then shifting

XPULP uses:

- **multi-operand adders**

Meaning:

- main value
 - rounding bias
 - carry-in
- are added **in one structure**

So:

- no extra cycles
- no extra adder blocks
- no timing penalty

This is why the slide explicitly says:

“Multiple adders for round”

5. Adder followed by shifter → fixed-point normalization

This is classic DSP design.

Fixed-point arithmetic often requires:

- compute
- normalize (shift left/right)

XPULP connects:

```
adder → shifter
```

directly, instead of:

- writing back
- re-reading
- re-shifting

Benefits:

- fewer pipeline stages
- less register traffic
- lower energy
- deterministic timing

So when you execute:

- add + normalize
- mul + normalize

You're **reusing the same datapath**.

6. Saturation & clipping: almost free

You already noticed this earlier — here's the hardware reason.

Clipping requires:

- comparison with min/max
- select output

XPULP does this by:

- reusing the **adder as a comparator**
- using the **main comparator** already present

So:

- no new arithmetic blocks
- just control + muxing

That's why the slide says:

“Clip unit uses one adder as comparator and the main comparator”

Area cost $\approx$ negligible

Energy cost $\approx$ negligible

But huge software simplification.

7. “No timing problems” — what that really means

This is subtle but important.

Even with:

- packed-SIMD
- wide shifters
- clip logic
- rounding

They ensure:

- **critical path does not grow**
- no extra pipeline stage required
- still meets near-threshold timing

How?

- no chained independent blocks
- reuse instead of stacking
- careful mux placement

This matters a lot at **low voltage**, where timing margins are tight.

8. Branch decision reuse

Notice at the bottom:

- `branch_decision` comes out of the same logic

Meaning:

- comparisons for DSP
- comparisons for branches
use the **same comparator hardware**

Again:

control logic is shared, not duplicated.

9. What this slide proves architecturally

This slide validates the whole XPULP story:

- ISA extensions are **hardware-aware**
 - Instructions were chosen because:
 - they reuse existing blocks
 - they don't lengthen critical paths
 - Performance gains come from:
 - *better datapath usage*, not brute force
-

10. One clean takeaway (interview / exam ready)

The XPULP ALU is a shared, multi-mode datapath where adders, shifters, and comparators are reused across DSP, packed-SIMD, clipping, and fixed-point operations. By carefully wiring the adder to the shifter and reusing comparators for saturation and branching, XPULP supports rich DSP semantics with minimal area, no extra pipeline stages, and no timing degradation—ideal for near-threshold operation.

Exactly — that short sentence “**we register the interrupt before jumping**” is doing a *lot* of work here. Let me unpack this slide cleanly and precisely, and tie it back to **RISC-V semantics** and **why PULP keeps it simple**.

1. Interrupts = asynchronous events

In PULP (and RISC-V in general):

- Interrupts are **asynchronous**
- They can arrive:
 - between any two instructions
 - regardless of the current control flow

Typical sources:

- peripherals
- timers
- DMA completion
- software events

2. What happens when an interrupt is taken (step-by-step)

When an interrupt is **taken**, the core does **three essential things** *before* jumping:

① Save the context (this is what you meant)

The core **registers the interrupt** by writing CSRs:

- `mepc`
→ saves the **PC of the interrupted instruction**
- `mcause`
→ encodes **which interrupt** occurred
- `mstatus`
→ updates interrupt enable bits (disables nested interrupts by default)

This is **mandatory** so that:

- the handler knows *why* it was called
- execution can resume correctly with `mret`

👉 This is what “register the interrupt before jumping” means.

② Compute the handler address using `xtvec`

The slide says:

If interrupt is taken, jump to `xtvec`

In RISC-V:

- `xtvec` (usually `mtvec`) holds a **base address**
- PULP uses **vectored mode**

So the actual jump address is:

```
handler_address = xtvec + 4 × interrupt_ID
```

Why $4 \times ID$?

- each handler entry is one 32-bit instruction wide
- constant-time dispatch
- no software decoding needed

This is **fast interrupt entry**, which matters for low-power MCUs.

③ Jump to the handler

Only **after** steps (1) and (2) does the PC jump to:

- the correct interrupt service routine (ISR)

So the sequence is:

```
interrupt arrives  
→ save context (CSRs)  
→ compute handler address  
→ jump to handler
```

3. “No delegation supported” — why this matters

The slide says:

No delegation supported
All interrupts are handled in machine mode

Meaning:

- PULP cores typically implement **machine mode only**
- No supervisor / user mode interrupt routing
- No `medeleg` / `mideleg`

Why?

- simpler hardware
- smaller CSR set
- lower power
- deterministic latency

This fits PULP’s role as:

a microcontroller / control core, not a general-purpose OS CPU

4. External interrupt controller (INTC)

Another key bullet:

External interrupt controller interact with peripheral subsystem and SW events

This means:

- peripherals do **not** interrupt the core directly
- they signal the **interrupt controller**
- the INTC:
 - prioritizes interrupts
 - assigns interrupt IDs
 - raises a single interrupt line to the core

Benefits:

- clean separation
- scalable number of interrupt sources
- software events can trigger interrupts too

This is especially important with:

- μ DMA
- cluster events
- timers
- GPIOs

5. Why PULP uses vectored interrupts

Compared to a single entry point:

Single entry (non-vectored)

- jump to one address
- software reads `mcause`
- software branches to handler

PULP vectored

- hardware jumps **directly** to handler
- no branch
- no decoding
- fewer cycles
- lower energy

This matches the overall PULP theme:

remove control overhead in hardware when possible

6. Minimal but complete interrupt flow (summary)

Putting it all together:

1. Interrupt arrives (asynchronous)
2. Core **registers the interrupt**:
 - saves PC (`mepc`)
 - saves cause (`mcause`)
3. Core computes:
4. $\text{target} = \text{xtvec} + 4 \times \text{interrupt_ID}$
5. Core jumps to handler
6. ISR runs
7. ISR executes `mret`
8. Core resumes from `mepc`

No delegation, no flags, no speculation — **simple and deterministic**.

7. One-sentence takeaway (exam-ready)

In PULP cores, interrupts are handled in machine mode only; when an interrupt is taken, the core first records the interrupted context in CSRs, computes the handler address using vectored `xtvec` + $4 \times \text{ID}$, and only then jumps to the ISR, ensuring fast, deterministic interrupt handling with minimal hardware complexity.

1. Big picture: what problem this protocol solves

Between:

- **Interrupt Controller (INTC)**
- **PULP Core**

there is an **asynchronous boundary**.

Why this matters:

- interrupts come from peripherals at arbitrary times
- the core may take a few cycles before it can jump
- during those few cycles, **priorities can change**

So PULP needs a **handshake**, not a simple “level signal”.

2. Up to 32 interrupts, each with an ID

Yes:

- There are **up to 32 interrupt sources**
- Each interrupt has a **fixed ID** (e.g. 0–31)

Key point (this answers your question):

The interrupt IDs are static (fixed in hardware), not programmable

Meaning:

- ID ↔ interrupt source mapping is **hardwired**
- You don’t dynamically remap IDs in software
- Peripheral X will *always* be interrupt ID N

This simplifies:

- hardware
 - vector table layout
 - timing guarantees
-

3. Why “static ID” is a design choice

Static IDs mean:

- `handler_address = xtvec + 4 × ID` is always valid
- no extra indirection tables
- no dynamic priority resolution in the core

This fits PULP’s goals:

- small area
- deterministic latency
- MCU-style behavior

Think **NVIC-like**, not Linux-style IRQ routing.

4. The asynchronous interrupt protocol (step by step)

Let's walk through the bullets **in order**, matching the diagram.

① Interrupt Controller asserts request

INTC → Core:

```
req = 1  
id  = 5    (example)
```

This means:

“Interrupt ID 5 is currently the highest-priority pending interrupt.”

② Core does NOT jump immediately

Important line from the slide:

The core takes few cycles before jumping

Why?

- finish current instruction
- save context (CSRs)
- compute vector address

During these cycles:

- **the interrupt controller may change the ID**
 - e.g. a higher-priority interrupt arrives

So the ID line is **not stable yet**.

③ Core *samples* the ID it will use

At a precise point:

- the core **latches** the interrupt ID
- this ID is used to compute:

$PC = x_{tvec} + 4 \times ID$

This latched ID is now **the taken interrupt**.

This is what the slide means by:

The core tells the interrupt controller which ID has been used

④ Core sends ACK with the taken ID

Core → INTC:

```
ack = 1  
id  = 5
```

Meaning:

“I am officially handling interrupt ID 5.”

⑤ Interrupt controller clears that ID

INTC:

- clears pending bit for ID 5
- does NOT clear other pending interrupts
- can later reassert `req` if others are pending

This avoids:

- lost interrupts
 - double servicing
 - race conditions
-

5. Why this handshake is necessary

Without this protocol:

- interrupt ID could change mid-entry
- wrong handler could be executed
- interrupts could be lost

With this protocol:

- exactly **one interrupt ID is taken**
- core and controller stay synchronized
- deterministic behavior

This is especially important in:

- low-power designs
- slow clocks
- near-threshold operation

6. What is static vs programmable (clear answer)

Static (hardwired):

- number of interrupts (≤ 32)
- interrupt IDs
- ID $\leftrightarrow$ peripheral mapping
- vector spacing (4 bytes)

Programmable (software):

- enable / disable interrupts
- priority masking (in controller)
- handler code
- `xtvec` base address

So when you heard:

“something about it is static not programmable”

That refers specifically to:

interrupt IDs and their mapping

7. One-sentence exam-ready takeaway

PULP supports up to 32 statically assigned interrupt IDs; an asynchronous handshake between the core and the interrupt controller ensures that a single, well-defined interrupt ID is latched, acknowledged, and cleared before the core jumps to `xtvec + 4 × ID`, guaranteeing deterministic and race-free interrupt handling.

CSR = Control and Status Register

In one clean sentence:

CSRs are special registers used by the CPU to control operation and report status to software.

In RISC-V (and PULP), CSRs are used for

Control

- enable / disable interrupts
- set interrupt vector base (`mtvec`)
- configure privilege behavior
- control low-power modes

Status

- record interrupt/exception cause (`mcause`)
 - store return address on interrupt (`mepc`)
 - hold processor state (`mstatus`)
-

How CSRs differ from normal registers

General Registers	CSRs
Used for data & computation	Used for control & system state
Accessed by all instructions	Accessed by special CSR instructions
Programmer-managed	Hardware + OS/firmware-managed

Examples of important CSRs in PULP

CSR	Purpose
<code>mtvec</code>	Interrupt vector base address

CSR	Purpose
<code>mepc</code>	PC to return to after interrupt
<code>mcause</code>	Which interrupt/exception occurred
<code>mstatus</code>	Global interrupt enable & state

Why CSRs matter for interrupts (tie-in)

When an interrupt happens, the core:

1. **writes CSRs** (records the event)
2. **jumps to handler**
3. handler reads CSRs to know what happened
4. `mret` restores state from CSRs

That's why earlier we said:

“we register the interrupt before jumping”

That “registration” is done **in CSRs**.

One-liner (exam / interview)

CSR stands for Control and Status Register, a class of special-purpose registers used to configure the processor and record events such as interrupts and exceptions.

Save power by clock gating while WFI , where we can see on the waveform the ALU,MULDOTP operands do not change when the units are not used which of-course cost more Area(More registers are needed to save operands) but ultimately dynamic power is saved.

You've got the **right interpretation**, and this slide is basically the *power-management counterpart* to the performance story you've been following. I'll explain it in a structured way and explicitly connect **WFI, clock gating, area vs power**, and the **waveforms**.

1. WFI (Wait For Interrupt): what it really does

WFI is not just “sleep” — it is a controlled clock-gating entry point.

When the core executes `WFI`:

1. The instruction reaches the commit stage
2. The core **waits for all in-flight instructions to complete**
 - pending loads
 - long divisions
 - FP / MAC operations
 - debug transactions
3. Only then:
 - the **core clock is disabled (gated)**
 - the core enters **IDLE**

This guarantees:

no partial state, no broken pipeline, no lost transaction

That's why the slide says:

The core waits for all the inflight instructions before switching off the clock

2. What wakes the core up

Any interrupt:

- taken **or even just pending**
- wakes the core up

Key detail:

- execution resumes at the **instruction after WFI**
- if the interrupt is enabled → interrupt handler is entered
- otherwise → normal execution continues

So WFI is **precise and restartable**.

3. Why WFI saves *dynamic* power

Dynamic power $\approx$

$$\left[\begin{array}{l} P \propto C \cdot V^2 \cdot f \cdot \text{switching activity} \end{array} \right]$$

WFI reduces:

- $f \rightarrow$ clock stopped
- switching activity $\rightarrow$ **zero**

But:

- state is retained
- registers keep values
- no leakage optimization here (that's power gating, not clock gating)

Hence:

Dynamic power saved when core is in IDLE

4. Clock gating beyond WFI (fine-grain gating)

The second half of the slide is even more important.

PULP does fine-grained clock gating, not just full-core gating.

The bullets say:

- pipeline and state registers are clock gated when not used
 - ALU, MUL, DOTP have **separate operand registers**
 - only one domain is active per instruction
-

5. Why separate operand registers exist

This is the key **area vs power** trade-off you mentioned.

Instead of:

- one shared operand register bank
- everything toggles every cycle

XPULP does:

- **ALU operands registers**
- **Multiplier operands registers**
- **Dot-product operands registers**

Each domain:

- has its **own registers**
- its **own clock enable**

So when an ALU instruction executes:

- ALU operand registers toggle
 - MUL + DOTP registers are **clock gated**
 - no useless switching
-

6. Interpreting the waveform (important)

From the waveform:

- Instruction stream alternates:
- ALU → MUL → ALU → DOTP → ...
- But:
 - `alu_operands` only change on ALU cycles
 - `mul_operands` only change on MUL cycles
 - `dotp_operands` only change on DOTP cycles

When a unit is not used:

- its operand registers **hold their value**
- clock is disabled
- **zero switching** → **zero dynamic power**

This is exactly what you described.

7. Why this costs more area (and why it's worth it)

Area cost:

- extra operand registers
- extra clock-gating logic
- extra control signals

Power benefit:

- large reduction in switching
- especially effective at:

- low frequency
- near-threshold voltage
- DSP workloads with mixed ops

PULP chooses:

a bit more area to dramatically reduce energy per operation

This is textbook **ultra-low-power design**.

8. Why this fits perfectly with PULP's philosophy

Everything connects now:

- ISA extensions → fewer instructions
- HW loops → fewer branches
- pSIMD → more work per cycle
- WFI → zero idle power
- fine-grain clock gating → minimal active power

No speculation.

No aggressive pipelines.

No wasted toggling.

9. One-paragraph, exam-ready summary

The PULP core reduces dynamic power by aggressively clock-gating both the entire core during WFI and individual execution domains during normal operation. Before entering WFI, the core waits for all in-flight instructions to complete, ensuring precise state retention. During execution, separate operand registers for the ALU, multiplier, and dot-product units allow only the active unit to toggle, while others remain clock gated. Although this increases area due to additional registers, it significantly reduces switching activity and thus dynamic power, which is critical for ultra-low-power and near-threshold operation.

here he talked that there are counters to count the cycles needed by an instruction , so no forwarding the operand in the pipeline until the operand is actually ready , the he spoke about the difference between jump and branch where jump is a taken branch (unconditional) , then there is the example

Great, this is the **last conceptual block** of the lecture: *observability*.
You already caught the essence; I'll clearly explain **what the counters are, what exactly is being counted, why stalls exist, and how the example works**.

1. What “performance counters” are in PULP

Performance counters are:

CSRs that count micro-architectural events

They are **not timers** and **not ISA-visible behavior**, but hardware probes.

Typical counters:

- cycle count
- retired instructions
- load stalls
- jump stalls
- branch stalls
- pipeline bubbles

These counters let you compute:

- **IPC** = instructions / cycles
- where time is lost (memory, control flow, data hazards)

2. “No forwarding until the operand is ready”

This sentence is key.

PULP cores are:

- simple
- energy-efficient
- limited forwarding logic

So for **loads**:

```
lw    x10, 0(x2)
add   x10, x10, x4
```

The `add`:

- needs `x10`
- but `x10` is **not valid until WB**

Therefore:

- pipeline **stalls**
- bubble is inserted
- forwarding cannot help

This stall is **explicitly counted**.

3. Load stall (slide 1/3)

What is counted

A **load stall** happens when:

- an instruction in ID/EX needs a register
- whose value is being loaded
- but memory has not returned it yet

The pipeline timeline shows:

- IF continues
- ID detects dependency
- EX inserts bubble
- WB finally gets data
- execution resumes

Each such bubble:

increments the **LD_STALL counter**

4. Jump stall (slide 2/3)

Jump vs branch (important distinction)

Instruction	Condition	Target known when
-------------	-----------	-------------------

Branch (<code>beq</code>)	Conditional	EX stage
-----------------------------	-------------	----------

Jump (<code>jalr</code>)	Unconditional	EX stage
----------------------------	---------------	----------

Even though `jalr` is *always taken*, the target address:

- depends on a register
- computed in EX

So the pipeline must:

- **stall IF**
- prevent fetching wrong instructions

This stall:

is counted as a **JALR_STALL**

5. Why PULP counts jump stalls separately

Because:

- branches depend on comparison
- jumps depend on address calculation

They stress **different pipeline paths**:

- EX → PC path
- EX → IMEM path

So they deserve **different counters**.

6. What “jump is a taken branch” really means

Conceptually:

- both redirect control flow
- both break sequential fetch

Microarchitecturally:

- branch may or may not be taken
- jump is **always taken**

But:

both cause pipeline disruption

Hence both can create stalls, but for different reasons.

7. The example (slide 3/3) — step by step

Code

```
csrw 0x782, x0      # reset LD_STALL counter

lp.setupi x1, 100, stop_loop

lw    x10, 4(x14!)   # load
stop_loop:
add    x11, x11, x10 # uses loaded value → stall

csrr  x15, 0x782     # read LD_STALL counter
```

What happens

Loop runs **100 times**.

Each iteration:

1. lw issues
2. add immediately depends on x10
3. load data not ready
4. pipeline inserts **1 stall**
5. counter increments

So:

LD_STALL = 100

Which is why:

x15 contains 100

This is **perfect correlation** between code structure and counter result.

8. Why this is powerful

With performance counters you can:

- verify pipeline behavior
- confirm compiler scheduling quality
- decide whether to:
 - reorder instructions
 - unroll loops
 - use DMA
 - use HW loops
 - use packed-SIMD

This is how you **tune for energy**, not just speed.

9. One-paragraph summary (exam-ready)

PULP performance counters are CSR-based registers that count micro-architectural events such as cycles, retired instructions, and pipeline stalls. Load stalls occur when an instruction depends on a load whose data has not yet returned from memory, while jump stalls occur because the jump target is computed in the EX stage, temporarily halting instruction fetch. By reading these counters, software can quantify pipeline inefficiencies and guide optimizations such as instruction reordering, hardware loops, and SIMD usage.

10. Final insight (ties whole lecture together)

Performance counters close the loop:

ISA extensions → microarchitecture → power → measurement

Without them:

- you guess

With them:

- you *prove* where cycles and energy go

Now we will talk about the logarithmic interconnect , which is responsible for arbitration , we have 64 bit AXI of the cluster if we are using a cluster , it has an increased bandwidth because it is expensive the transfer data from and to different clock and voltage domains so every access from the cluster is 64 bit not 32, we have the other masters some are read only some are read write , the HWCE is x4 as it is 128b it , in the interleaved banks data is organized horizontally while in the private banks data is organized vertically Most of L2 banks are interleaved as in DSP applications we are accessing vectors so different banks in cycles as for multiple master they can be working on the same data reading and storing

results .So, we use interleaved memory to decrease probability of bank conflicts and stalls .he says interleaving banks is a different way to emulate multi-ports memory cards with the difference the the first one(Multi-port memory cards)never stalls (ex processing it and transferring it to the UART) (, Private banks are not literally private micro-DMA can access them .they are called that so in the linker we map the data stack and the instruction of the core to them. which further decreases the probability of conflict where we can be running an algorithm in the private banks and simultaneously running a data transfer in the interleaved banks. he said LIC is single-access bus as most of the time is spent in the memory access not the bus so no real gain from pipelining it. every master can access every data in these memories. There are all tightly coupled (do not know what that means) they are not caches all are scratchpad (do not know that either) as for the synchronization between cores there is an event unit in the cluster that takes care about synchronization and all multithreading events and semaphores can be implemented. The masters are the core slaves they are driven by the core , no cache for energy efficiency to remove cache coherence overhead in the multicore cluster. Logarithmic so the number of stages is logarithmic to the number of masters and slaves . AT every stage the arbitration happens as well and based on the request it is either routed to the private banks or the interleaved one. with no stalling we can work on 128 bit per cycle with the interleaved structure as if we have 4 write and read ports, the stalling is working in the round robin fashion if once x work and Y stalled next time they will be reversed.

1. What is the *Logarithmic Interconnect (LIC)*?

The **LIC** is the **on-chip network** that connects:

- **masters** (cores, uDMA, HWCE, etc.)
- to **slaves** (L2 memory banks, peripherals, private banks)

“**Logarithmic**” means:

The number of arbitration/routing stages grows as **$O(\log N)$** instead of $O(N)$

So if you double the number of masters or slaves, the interconnect depth increases **slowly**, keeping:

- latency reasonable
- wiring complexity manageable
- arbitration scalable

This matters a lot in **clusters** with many cores + accelerators.

2. Why a 64-bit AXI interface for the cluster?

You caught the key reason 🙌

Crossing **clock and voltage domains** is expensive.

So PULP chooses:

- **64-bit AXI** between cluster ↔ SoC
- instead of 32-bit

Why?

- Fewer transfers for the same data
- Lower protocol overhead
- Better energy efficiency per byte

Inside the cluster:

- accesses are **wide and local**
 - latency matters more than raw frequency
-

3. Masters, slaves, and bandwidth asymmetry

Masters

- CPU cores (R/W)
- uDMA (mostly burst R/W)
- HWCE (heavy R/W, 128-bit wide)
- Debug, peripherals, etc.

Slaves

- L2 **interleaved banks**
- L2 **private banks**
- peripheral registers

Not all masters are equal:

- some are **read-only**
- some **read-write**
- some very **bandwidth hungry** (HWCE)

That's why arbitration exists at **every LIC stage**.

4. Why HWCE is “×4” (128-bit)?

HWCE processes **vectors** / **convolution windows**, so:

- $4 \times 32\text{-bit}$ words per cycle
- = **128-bit datapath**

Instead of slowing HWCE down:

- memory system adapts to it
- interleaved banks can deliver **one word per bank per cycle**

This is where interleaving shines.

5. Interleaved banks vs private banks (core idea)

Interleaved L2 banks

- Data is laid out **horizontally**
- Consecutive words go to **different banks**

Example (4 banks):

```
A[0] → bank 0  
A[1] → bank 1  
A[2] → bank 2  
A[3] → bank 3  
A[4] → bank 0  
...
```

☒ Perfect for:

- vectors
- DSP kernels
- multiple masters accessing adjacent data

☐ Emulates **multi-port memory**

☐ But with **possible conflicts**

Private L2 banks

- Data is laid out **vertically**
- One bank per core (logically)

Used for:

- stack
- instructions
- private variables

Important correction (you got it right):

“Private” $\neq$ inaccessible
uDMA **can access them**

They are “private” only at **linker / software level**.

6. Why interleaving reduces bank conflicts

Without interleaving:

- vector access $\rightarrow$ same bank $\rightarrow$ stalls

With interleaving:

- vector access $\rightarrow$ round-robin banks
- multiple masters likely hit **different banks**

So:

- lower stall probability
- higher sustained bandwidth

That’s why:

“Most of L2 is interleaved in DSP workloads”

7. Interleaving vs true multi-port memory

Multi-port SRAM Interleaved banks

Never stalls	Can stall
Very expensive	Much cheaper
Hard to scale	Scales well
High area & power	Area-efficient

So interleaving is:

A **cost-effective approximation** of multi-port memory

8. Why LIC is a single-access bus

This is subtle but important.

Most time is spent **inside the memory**, not on the bus.

So:

- pipelining the bus gives little benefit
- arbitration + routing dominates anyway

Hence:

- **single access per stage**
 - simpler design
 - lower energy
-

9. “Tightly coupled” — what does that mean?

Tightly coupled means:

- low latency
- deterministic access

- no speculative behavior

In PULP:

- cores directly access L2
 - no cache miss penalties
 - predictable timing (important for DSP & RT)
-

10. “Scratchpad memory” — what does that mean?

A **scratchpad** is:

- software-managed memory
- NOT a cache
- no tags, no coherence, no eviction

Programmer/compiler decides:

- what goes where
- when it moves

Benefits:

- deterministic timing
- no cache coherence overhead
- much lower energy

That’s why:

PULP **avoids caches** inside the cluster

11. Synchronization: Event Unit

Instead of cache-based atomics:

- PULP uses an **Event Unit**

It supports:

- barriers

- events
- wake-ups
- mutex-like behavior

Perfect for:

- multicore DSP
 - fork/join parallelism
 - low-power waiting
-

12. Arbitration & fairness (round-robin)

At each LIC stage:

- requests are arbitrated
- round-robin fairness

If:

- X wins, Y stalls
Next time:
- Y wins, X stalls

This prevents:

- starvation
 - bandwidth monopolization
-

13. 128-bit per cycle “as if 4 ports”

Exactly right.

With:

- 4 interleaved banks
- each 32-bit wide

You effectively get:

- 128-bit per cycle

- **if no conflicts**

So:

Looks like 4-port memory

Behaves like 1-port per bank

14. Grammar side question 😊

? “*data is*” or “*data are*”

- **Formally:** *data are* (plural of datum)
- **Engineering usage:** *data is* (mass noun)

Both are accepted — “**data is**” is common in technical writing.

15. One-paragraph summary (exam-ready)

The PULP logarithmic interconnect connects multiple masters and memory banks using a scalable arbitration network whose depth grows logarithmically. Interleaved L2 banks distribute consecutive words across banks to emulate multi-port memory and reduce conflicts in vector-based DSP workloads, while private banks isolate core stacks and instructions. The system avoids caches in favor of scratchpad memories for deterministic timing and energy efficiency. Arbitration is fair and round-robin, and wide datapaths (64-bit AXI, 128-bit HWCE) amortize domain-crossing costs while sustaining high bandwidth.

This is a single master bus (The peripheral interconnect is a bus) where the Master is the LIC , once we access the APB the peripheral bus connect us to the peripheral domains we want to access , list the masters of LIC interconnect the clusters the micro DMA and/or what else

1. First: what this slide is really saying

Yes ☒

The **Peripheral Bus (APB)** is a **single-master bus**.

And also yes ☒

That **single master is the LIC** (Hybrid Logarithmic Interconnect).

So:

Peripherals never arbitrate among themselves

Arbitration already happened *before* the request reached APB.

That's the key architectural idea.

2. Role separation (very important)

Inside the chip:

- **LIC** = arbitration + routing fabric
- **APB** = simple peripheral bus (no arbitration)

So the flow is:

Multiple masters → LIC (arbitrate) → 1 APB request → peripheral

If more than one APB request exists:

- they stall **inside the LIC**
 - exactly like a **bank conflict**
-

3. Masters of the LIC (this is what you asked)

The **LIC masters** are all blocks that can *initiate memory or peripheral transactions*.

◇ Main LIC masters in PULP SoC

1. Cluster interface

- The **entire cluster** appears as one or more masters
- Internally:
 - cores
 - HWCE
 - cluster uDMA
- Externally:

- exported as **64-bit AXI master**

So:

Cluster $\neq$ peripheral

Cluster = heavy master

2. uDMA (SoC-level micro-DMA)

- One of the most important masters
- Initiates:
 - peripheral $\leftrightarrow$ L2 transfers
 - memory $\leftrightarrow$ memory copies
- Accesses:
 - L2 interleaved banks
 - L2 private banks
 - peripherals (through APB)

So yes:

micro-DMA is a master of the LIC

3. Fabric Controller (FC) core

- The main RISC-V core outside the cluster
 - Acts as a **general-purpose master**
 - Used for:
 - boot
 - OS
 - control code
 - peripheral configuration
-

4. Debug / JTAG

- Debug access can initiate memory reads/writes
 - Treated as a **low-priority master**
 - Still goes through LIC
-

5. HW accelerators (SoC-level, if present)

Examples:

- HWCE (if outside cluster)
- crypto engines
- FFT accelerators

These are:

- bandwidth-hungry
 - often wide (64/128-bit)
 - full LIC masters
-

4. What is NOT a master

Important clarification:

✗ Peripherals are NOT masters

- GPIO
- SPI
- I2C
- Timer
- Event unit
- RTC

They are **APB slaves** only.

They *respond* — they never initiate.

5. Why APB is single-master (by design)

APB is:

- low-frequency
- low-power
- register-oriented

Peripherals:

- don't need high bandwidth
- don't need parallel accesses

So:

- **no arbitration**
- **one request at a time**
- very cheap logic

That's why:

“Only one APB request; if more, stalled in LIC”

6. Address decoding example (0x1A1_00000)

This address:

- falls into the **peripheral address space**
- LIC decodes it
- routes it to **APB**
- APB selects:
 - GPIO
 - SPI
 - I2C
 - etc.

The peripheral bus is just:

address → decoder → peripheral

7. Mental model (exam / interview friendly)

The LIC is a multi-master interconnect that arbitrates and routes requests from cores, DMA engines, clusters, and accelerators. When a request targets peripherals, the LIC forwards it to a single-master APB bus, ensuring that peripherals remain simple and energy-efficient while contention is handled upstream.

8. One-line answer to your question

Masters of the LIC include:

- Fabric Controller core
- Cluster interface (cores + HWCE + cluster DMA)
- SoC micro-DMA (uDMA)
- Debug/JTAG
- Hardware accelerators

APB has exactly one master:

- the LIC itself

Micro-DMA autonomously transfers data to support above 1 GHz bandwidth , it is programmable specify number of transactions, freq , the side near peripheral is low speed as it speaks the peripheral language , they can all work concurrently as each has buffer and configuration registers whatsoever controlling them and there is only two ports to the logarithmic interconnect so it is N:2 reduction in bandwidth , another model that pretends to be a peripheral but actually is a filter a tiny accelerator inside the micro DMA which is for instance doing the fear filter of the data coming from the SPI(fear filter?!!) it is an accelerator that is working with peripheral data thus it does need to be as fast as an accelerator that has the full bandwidth of the L2 so it shares the bandwidth with peripheral which are slow so they do not need the full bandwidth therefore you can save some ports towards the L2 as the filter is using the micro DMA one (A slow accelerator inside the subsystem)

1. What the uDMA subsystem really is

Think of **uDMA** as a **programmable data-mover fabric**, optimized for **high aggregate bandwidth** with very low core involvement.

Its job:

- Move data **between peripherals and L2**
- Without waking cores
- With **predictable bandwidth**
- At **much lower energy** than CPU load/store loops

Key idea:

uDMA trades *peak per-channel bandwidth* for *massive concurrency and efficiency*.

2. Why “micro”-DMA?

It's called **micro-DMA** because:

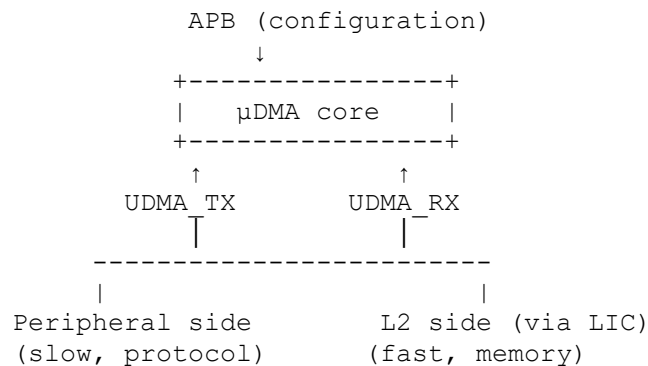
- It is **simple**
- It is **fully programmable**
- It handles **small, frequent transfers**
- It is tightly integrated with peripherals

Unlike a big SoC DMA:

- No scatter-gather engine
- No virtual memory
- No cache coherency headaches

Perfect for **embedded streaming workloads**.

3. High-level structure (reading the slide)



4. Two faces of uDMA (this is crucial)

◇ Peripheral-facing side (slow, many)

- Speaks **peripheral protocols**:
 - SPI
 - I2S

- UART
 - Camera
- Each peripheral has:
 - its own **config registers**
 - its own **FIFO/buffer**
- Clocked at **peripheral speed**

👉 This side is **wide in number**, but **slow in bandwidth**.

◇ Memory-facing side (fast, few)

- Only **two ports** to the L2:
 - RX port
 - TX port
- These ports go to:
 - L2 interleaved banks
 - L2 private banks

This is where:

N : 2 bandwidth reduction happens

And that's intentional.

5. Why only 2 ports to L2? (important design decision)

You nailed the reasoning 👉

Reality:

- Peripherals are **slow**
- SPI, UART, I2S cannot saturate L2 bandwidth
- Even many of them together rarely hit L2 limits

So instead of:

✗ Giving each peripheral its own L2 port (huge area & power)

They do:

- ☒ Aggregate them inside uDMA
- ☒ Share **2 high-bandwidth ports**

This saves:

- area
 - wiring
 - power
 - arbitration complexity
-

6. Concurrency: “they can all work at the same time”

Correct ☒

Each uDMA channel:

- has independent config registers
- has independent FIFOs
- runs autonomously

So:

- SPI RX
- UART TX
- Camera RX
- I2S TX

can **all be active simultaneously**, even though they share only:

- 1 RX port
- 1 TX port to L2

The uDMA core schedules them.

7. The “filter” block (not “fear filter” 😊)

You were *very close*.

This is **NOT a FIR filter**.

It is:

a **lightweight streaming accelerator** inside the uDMA subsystem

What it really is:

- A **data-path modifier**
 - Works *on the stream*, not on memory
 - Examples:
 - unpacking
 - packing
 - format conversion
 - decimation
 - simple DSP pre-processing
-

8. Why put an accelerator inside uDMA?

This is a **brilliant architectural trick**.

Normal accelerator:

- Needs full L2 bandwidth
- Needs wide ports
- Needs arbitration priority

uDMA filter:

- Operates at **peripheral speed**
- Shares uDMA's slow side
- Never stresses L2
- Saves L2 ports

So yes, exactly as you said:

“It pretends to be a peripheral but is actually a tiny accelerator”

This is **stream processing**, not bulk compute.

9. Why the filter does NOT need full L2 bandwidth

Because:

- Input rate = peripheral rate

- Output rate = peripheral rate
- Peripheral rate $\ll$ L2 bandwidth

So giving it:

- a dedicated L2 master would be wasteful.

Instead:

- It reuses the uDMA's existing ports
- Zero extra pressure on LIC

10. Why this is different from cluster accelerators

uDMA filter	Cluster accelerator
Stream-based	Memory-based
Peripheral-speed	L2-speed
Low area	High area
No cache	No cache
No coherency	No coherency

Both are accelerators — **but at different bandwidth tiers.**

11. End-to-end example (SPI + filter)

1. SPI receives data
2. uDMA RX channel captures it
3. Filter processes stream (e.g. unpack / scale)
4. Result written to L2
5. Core is asleep (WFI)
6. Event wakes core when buffer full

Zero CPU cycles wasted.

12. One-sentence summary (exam gold)

The uDMA subsystem aggregates many low-speed peripheral data streams and autonomously transfers them to L2 through two shared high-bandwidth ports, optionally performing lightweight stream processing via embedded filters, thereby maximizing concurrency while minimizing area and power.

i did not catch much it is an example showing the flow and interrupts send , it is pipelined fashion , explain double buffering

1. What this slide is showing (big picture)

This is **not** about a single transfer.

It shows a **streaming pipeline** where:

- I/O keeps producing data
- DMA keeps moving data
- an accelerator keeps processing data
- the CPU only **coordinates**, it does not touch the data

All of this happens **in parallel**.

2. The actors (top → bottom)

From the slide:

I/O

- Peripheral (SPI, camera, I2S, etc.)
- Produces a continuous data stream

CPU

- Programs the DMA
- Receives interrupts
- Starts the next transfer
- Mostly sleeps (WFI)

Acc. DMA

- Copies data from **L2** → **accelerator local memory**

Acc. Processing

- The accelerator actually computes on the data
-

3. The key concept: double buffering

What is double buffering?

Using two buffers so that one is being filled while the other is being processed.

Instead of:

✗ fill → wait → process → wait → fill → wait

You do:

- ☒ fill buffer A while processing buffer B
 - ☒ then swap
-

4. Why single buffering is bad (intuition)

With **one buffer**:

Time →

[RX data] [PROCESS] [RX data] [PROCESS]

- Peripheral idle during processing
- Accelerator idle during reception
- CPU forced to synchronize constantly

Lots of wasted time ✕

5. Double buffering timeline (what the slide shows)

Let's name the buffers:

- Buffer0
- Buffer1

Step-by-step:

Phase 1

- I/O fills **RX Buffer0**
- Nothing else yet

Phase 2

- CPU gets interrupt: "Buffer0 full"
- CPU programs DMA:
 - copy Buffer0 → accelerator
- I/O **immediately starts filling Buffer1**

Now things overlap 🔥

Phase 3

- DMA copies **Buffer0**
- Accelerator processes **Buffer0**
- I/O fills **Buffer1**

All three active at once.

Phase 4

- Buffer1 full → interrupt
- CPU programs DMA for Buffer1
- I/O goes back to Buffer0
- Accelerator switches to Buffer1

This repeats forever.

6. What “offload pipeline” means

Each stage is offloaded to hardware and runs independently.

Stage	Who does it
I/O	Peripheral
Transfer	uDMA
Copy to accelerator	Accelerator DMA
Compute	Accelerator
Control	CPU

The CPU is **not in the data path**.

7. Where interrupts fit in

Interrupts are used **only at buffer boundaries**:

- RX buffer full
- DMA transfer done
- Accelerator done

So interrupts are:

- infrequent
- deterministic
- cheap

The CPU wakes up just long enough to:

- program next DMA
- then goes back to sleep (WFI)

8. Why this is pipelined

Because different **buffers** are at different **stages** simultaneously:

```
Time →  
Buffer0: RX → DMA → PROCESS  
Buffer1:          RX → DMA → PROCESS
```

That is literally a **pipeline**, but across **subsystems**, not CPU stages.

9. Why this is extremely efficient

Performance

- No bubbles between stages
- Throughput limited only by slowest stage

Energy

- CPU mostly sleeping
- No polling
- No cache pollution

Scalability

- Multiple streams can be time-multiplexed
 - Same idea works for:
 - camera
 - audio
 - sensors
-

10. One-sentence definition (important)

Double buffering allows continuous data streaming by overlapping data acquisition, transfer, and processing using two alternating buffers, eliminating idle time between pipeline stages.

11. Why PULP emphasizes this design

Because PULP targets:

- DSP
- sensor processing
- edge AI
- streaming workloads

And in those:

Throughput matters more than single-instruction latency
